

Assignment 1, Week 1

Hans Daruka, Roll No. – 25B1064

1.1)

a) Sequence of Kernel actions is as follows:

Process P1 was RUNNING when hardware timer fires ->

Kernel interrupt handler ->

Save P1 context into PCB ->

P1: RUNNING - READY ->

P1 is inserted into run queue ->

Scheduler selects another process P2 which was READY ->

P2: READY - RUNNING ->

Restore P2 context from PCB ->

Returns from interrupt ->

P2 resumes execution

b)(i) First invalid transition is Running -> Terminated

Calling read() on an empty process does not terminate the process, instead it blocks the process waiting for data.

Second invalid transition is Terminated -> Blocked

After termination, a process dies, it cannot change its state ever again (the scheduler never runs it). So a terminated process cannot be blocked.

(ii) The correct transition sequence is-

Running -> Blocked -> Ready -> Running

- Step 1 is initiated by the process itself (process calls read())
- Step 2 is done by hardware interrupt (when data arrives)
- Step 3 is done by OS scheduler

c) A zombie process still occupies an entry in the process table even after exiting because the kernel must preserve its exit status and termination information until the parent collects it. Although the process is no longer executing, its PCB entry remains.

The zombie is removed when the parent calls `wait()`, and the PCB entry and PID is released.

If the parent exits before collecting the child's exit status, the zombie is adopted by the `init` process which can later remove it by calling `wait()`.

1.2)

a) 2^n processes, this is because the number of processes doubles on each iteration (as child is not terminated inside the loop). Every existing process creates one additional process at each `fork()`.

In the given code, only 3 processes are created because second `fork()` is run inside the block (if `rc1==0`) so a third process is created only by the first child process.

b) C: `g=110 x=105`

B: `g=10 x=5`

A: `g=10 x=5`

Whenever a child process is created, it makes its own copy of memory of the parent, so any modifications made within the child process does not propagate to its parent.

c) Any order other than C->B->A is impossible because of `wait()` commands.

d)(i) No, even the heap memory is duplicated after calling `fork()`.

(ii) The use-after-free bug can occur in such case. It occurs when memory is accessed after being deallocated.

Explanation – Copy-on-Write (CoW): It is a resource management technique where the system delays duplicating data until a modification is requested. Instead of copying data immediately, the original and the new process share the same memory.

(iii) Process C becomes an orphan process but it continues to run normally. In such case, the OS re-parents C to the `init` process.

If this situation happens repeatedly, zombie processes may accumulate and exhaust the resources of PCB.

2.1)

a)(i) Only at $t=2\text{ms}$, preemption occurs (P1 preempts P2), at all other instants when a new process arrives, it is just a scheduling decision of SRTF as the already running program finishes at the same time.

(ii) Yes, P1 can finish last under SRTF because it has the largest burst time. Whenever shorter jobs arrive, they preempt or delay P1.

P1 will definitely finish last if every arriving process has time smaller than P1's remaining time.

Impossible for P1 to finish last if at some time, P1 becomes shortest remaining process.

b) Under FCFS, processes execute strictly in arrival order:

$P1 \rightarrow P2 \rightarrow P3 \rightarrow P4 \rightarrow P5$

Since P1 has a long burst time (10 ms), all later processes including very short jobs like P4 (2 ms) must wait behind it and behind other longer jobs already queued.

Under non-preemptive SJF, once P1 completes, the scheduler chooses the shortest available job first. Since all remaining processes have arrived by then, the execution order becomes approximately:

$P4 \rightarrow P2 \rightarrow P3 \rightarrow P5$

This reduces the waiting time of short processes, especially P4 and P2, which are the primary contributors to the improvement.

Therefore, SJF must produce a lower (or equal) average wait time than FCFS for this workload because it prioritizes shorter jobs instead of forcing them to wait behind longer ones.

c) Avg. wait time = 12.2ms

Avg. turnaround time = 18.2 ms

Fairness ensures lesser response time and not turnaround time.

d) Avg. wait time = 8ms

Avg. turnaround time = 14 ms

2.2)

a) Avg. Wait time for SJF = $0 + b_1 + (b_1 + b_2)/3 = (2b_1 + b_2)/3$

For any preemptive process, avg. wait time = $0 + b_i + (b_i + b_j)/3 = (2b_i + b_j)/3$

Which is minimum for $i=1$ and $j=2$ as $b_1 \leq b_2 \leq b_3$

b) (i) SRTF: avg response time = $0 + s + 2s + \dots + (k-1)s + ks/k+1 = ks/2$

RR: avg response time = $0 + q + 2q + \dots + (k-1)q + kq/k+1 = kq/2$

RR always produces a lower average response time than SRTF for $q < s$.

3.1)

a) wait time = start time

turnaround time = finish time

b) 30

c) $J = 0.746$

d) In SJF, wait times become more balanced, so less variance means higher J. So SJF reduces the large waits suffered by later short jobs under FCFS.

3.2)

a) SRTF, because many small jobs finish almost instantly and long jobs get postponed repeatedly.

Example:

P - arrival, burst

1 - 0, 100

2 - 1, 1

3 - 2, 1

So, P2 and P3 have 0 wait time while P1 waits for as long as these shorter processes keep coming.

b) A scheduler can minimize Makespan by maximizing CPU utilization and completing large amounts of work continuously, even if this delays many smaller processes. Thus the total workload finishes earlier, producing a lower Makespan, while individual processes may experience longer waits, increasing average wait time.

Scheduler Gamma likely favours throughput-oriented execution, reducing total completion time but sacrificing responsiveness for some jobs.

Makespan is generally not a good metric for interactive workloads because users care about latency and responsiveness rather than total batch completion time. An interactive system may have excellent Makespan while still producing noticeable delays for individual user requests.

c) For (i), I will surely choose gamma as it has lowest Makespan thus reducing the total throughput.

For (ii), I will choose Delta because for a real-time chat application latency/responsiveness is the most important thing, and the user always looks at the worst-case scenario. So delta is best as it has lowest P90 and P99 wait times.